

# INF01151 – Sistemas Operacionais II

---

## Aula 03 - Exclusão mútua

Prof. Alberto Egon Schaeffer Filho

---

# Dois problemas para aquecimento...

1. Implementar em pseudo-código um programa que permita a operadores de viagem reservar e emitir bilhetes aéreos:
  - Faça apenas a função para reserva de assentos e emissão do bilhete, para um determinado assento
2. Usando a notação vista na aula passada, diga qual é o comportamento esperado para o seguinte trecho de programa:
  - Descreva todas as suas suposições

```
continue = true;  
co while(continue);  
// continue = false;  
oc
```

Tempo para a atividade: 10 minutos

# Seção crítica

- Problema da reserva (compra) de uma passagem aérea
  - Agentes de viagem executam um código do tipo:

```
void reserveSeat(Position p) {  
    if (seat.free(p))  
        seat.reserve(p)  
}
```

- ...emitindo um bilhete aéreo para uma poltrona p

# Execução possível

Operador A

```
void reserveSeat(Position p) {  
    → if (seat.free(p))  
        seat.reserve(p)  
}
```

Operador B

```
void reserveSeat(Position p) {  
    → if (seat.free(p))  
        seat.reserve(p)  
}
```

# Qual é o valor final de counter?

- Considerando o trecho de código:

```
counter = 5;  
co counter++;  
// counter--;  
oc
```

counter++ <==>

```
MOV R1, counter;  
INC R1;  
MOV counter, R1;
```

counter-- <==>

```
MOV R2, counter;  
DEC R2;  
MOV counter, R2;
```

## Execução 1:

```
T0: MOV R1, counter;  
T1: INC R1;  
T2: MOV counter, R1;  
T3: MOV R2, counter;  
T4: DEC R2;  
T5: MOV counter, R2;
```

## Execução 2:

```
T0: MOV R2, counter;  
T1: DEC R2;  
T2: MOV counter, R2;  
T3: MOV R1, counter;  
T4: INC R1;  
T5: MOV counter, R1;
```

## Execução 3:

```
T0: MOV R1, counter;  
T1: INC R1;  
T2: MOV R2, counter;  
T3: DEC R2;  
T4: MOV counter, R1;  
T5: MOV counter, R2;
```

# Qual é o valor final de counter?

- Considerando o trecho de código:

```
counter = 5;  
do counter++;  
// counter--;  
oc
```

counter++ <==>

```
MOV R1, counter;  
INC R1;  
MOV counter, R1;
```

counter-- <==>

```
MOV R2, counter;  
DEC R2;  
MOV counter, R2;
```

Execução 1:

```
T0: MOV R1, counter;  
T1: INC R1;
```

Execução 2:

```
T0: MOV R2, counter;  
T1: DEC R2;
```

Execução 3:

```
T0: MOV R1, counter;  
T1: INC R1;
```

**Condição de corrida:** “Situação onde diferentes processos acessam e manipulam o mesmo dado concorrentemente e o resultado da execução depende da ordem em que os acessos acontecem.”

**Como evitar?** Sincronização de processos!

```
T2: MOV counter, R1;  
T3: MOV counter, R2;  
T4: MOV counter, R1;  
T5: MOV counter, R2;
```

# Introdução

- Unidades de execução: processo ou *thread*
  - Existe concorrência sempre que houver mais de uma unidade de execução no sistema
    - Concorrem (disputam) o recurso CPU (pelo menos)
- Execução independente ou cooperativa/competitiva
  - Se cooperativa/competitiva, duas ou mais unidades de execução necessitam interagir
    - Comunicar e sincronizar com seus pares

**IMPORTANTE:** Daqui para frente, o discurso é válido tanto para processos como para threads

# Estados de um programa concorrente

- Estado de programa concorrente
  - Conjunto de valores das variáveis do programa em um instante de tempo
    - **Implícitas**: contador de programa (PC), apontador de pilha (SP), etc
    - **Explícitas**: variáveis declaradas pelo próprio programador
  - Programa inicia execução em algum estado inicial  $S_0$
  - Cada processo do programa concorrente executa independentemente
    - Examina e altera o estado do programa
- Cada processo
  - Executa sequência de comandos
    - Comando = sequência de uma ou mais **ações atômicas** (indivisíveis), que examinam ou modificam estado do programa sem interrupções
    - Instruções de máquina que carregam ou armazenam palavras de memória são ações atômicas



# Programa concorrente

- Execução de um programa concorrente
  - Resulta na intercalação de sequências de ações atômicas executadas por cada processo

P1:  $C_{11} \rightarrow C_{12} \rightarrow C_{13} \rightarrow \dots \rightarrow C_{1n}$

P2:  $C_{21} \rightarrow C_{22} \rightarrow C_{23} \rightarrow \dots \rightarrow C_{2n}$

- Uma execução em particular de um programa concorrente pode ser vista como um histórico

*histórico*:  $S_0 \rightarrow S_1 \rightarrow S_2 \rightarrow \dots \rightarrow S_n$

- Partindo de um estado inicial,  $S_0$
- Cada estado subsequente é atingido por uma ação atômica que altera o estado

# Programa concorrente

- Execução de um programa concorrente
  - Resulta na **intercalação de sequências de ações atômicas** executadas por cada processo
  - Cada intercalação irá produzir um histórico diferente
- Número de combinações possíveis de intercalações é enorme!!
  - Algumas intercalações podem ser **indesejáveis!!!**
- Objetivo
  - Limitar o número de históricos possíveis, eliminando os indesejáveis

**Como fazer?**

Sincronização de processos!

# Sincronização de processos

- **Exclusão mútua**

- Composição de ações atômicas, implementadas diretamente por hardware, em um bloco de ações chamado seção crítica
- Seção crítica:
  - Conjunto de ações de um processo não podem ser intercaladas com ações de outros processos que também referenciem as mesmas variáveis
    - Ações “parecem” ser atômicas
    - Garante acesso exclusivo (sequencial) ao recurso

- **Sincronização condicional**

- Tarefa só pode prosseguir depois que uma condição seja satisfeita
  - Permite “atrasar” a execução de uma tarefa

- **Outros tipos... Barreira [mecanismo não tão popular]**

- Todas as tarefas sincronizam em um determinado ponto
  - As 1<sup>as</sup> tarefas esperam pelas últimas na barreira

# Propriedades de programa concorrente

- Propriedade
  - Atributo que é **verdadeiro em todos os históricos** possíveis do programa
    - Portanto, em todas as suas execuções
- Dois tipos de propriedades de programas
  - **Safety property**: programa nunca entra em um estado ruim
    - Estado onde algumas variáveis possuem valores indesejáveis
  - **Liveness property**: programa chegará a um estado bom
    - Estado onde variáveis tem valores desejáveis

Safety property

## Exclusão mútua

Estado ruim é aquele em que dois processos estão executando ações nas seções críticas simultaneamente

Liveness property

## Entrada na seção crítica

Estado bom para cada processo é aquele em que processo terá chance de estar executando dentro da seção crítica

# Ações atômicas

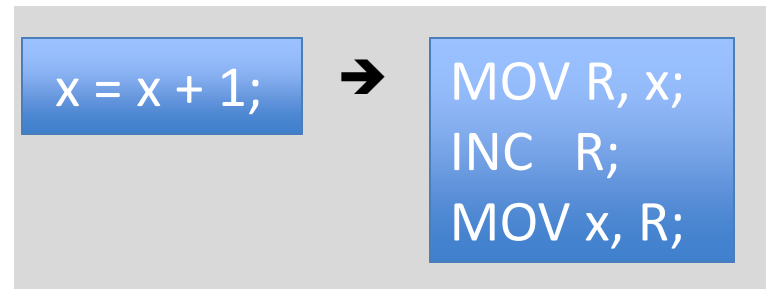
- Ação indivisível de uma tarefa: estados intermediários da ação não percebidos por outras tarefas
  - **Atomicidade fina**
    - Algumas ações são atômicas por hardware, por exemplo
      - Leitura e escrita de uma palavra de memória
  - **Atomicidade grossa**
    - Ações de sw normalmente não são atômicas
    - Se atomicidade for necessária
      - Uso de mecanismos de sincronização de exclusão mútua
      - Sequência de ações com atomicidade fina (hw) tornada atômica por mecanismos de sincronização
- Questão: verificar quando atomicidade grossa é necessária
  - Serializa tarefas, reduzindo vantagens da prog. concorrente

# Ações atômicas

- Atribuição de valores parece ser atômica, mas geralmente não é
  - Implementado através de sequência de instruções de máquina

## Valores possíveis de x?

```
int y = 0, z = 0;  
co x = y+z; // y = 1; z = 2; oc;
```



- Modelo de processador
  - Valores de tipos básicos de dados (p. ex. int) são armazenados em memória e lidos ou escritos de forma atômica
  - Para manipulação de valores, estes são carregados em registradores, manipulados, e armazenados na memória
  - Cada processo tem seu conjunto de registradores (lógico), salvo e restaurado através de chaveamento de contexto
  - Valores intermediários de expressões complexas são armazenados em registradores ou em posições de memória (pilha) privadas

# Avaliação de expressões

- Com o modelo de máquina apresentado...
- Se expressão *e* em um processo não fizer referencia à variável alterada por outro processo, avaliação irá “*parecer*” atômica
  - Mesmo que necessite executar várias operações de máquina
  - Isto porque
    - Nenhum dos valores em que *e* depende poderia ter sido alterado
    - Nenhum outro processo pode ver valores temporários que possam ser criados durante avaliação da expressão
- Restrição forte
  - Porém, existe um requisito mais relaxado...

# Propriedade at-most-once

- Referência crítica:
  - Variável em uma expressão  $e$  que pode ser modificada por outro processo
    - Suponha que variáveis são armazenadas em memória, lidas/escritas atomicamente
- Propriedade **at-most-once** em  $x = e$ 
  1. A expressão  $e$  possui no máximo uma referência crítica e  $x$  não é lido por outro processo

ou

  1. A expressão  $e$  não possui qualquer referência crítica, e  $x$  pode ser lido por outro processo



Valores lidos dependerão da ordem de execução, mas serão sempre valores válidos!



# Propriedade at-most-once: exemplos

Se propriedade for satisfeita, execução de atribuição parecerá ser atômica...

```
int x = 0; y = 0;  
co x = x + 1;  
// y = y + 1;  
oc OK
```

Sem referências críticas em qualquer processo

```
int x = 0; y = 0;  
co x = y + 1;  
// y = y + 1;  
oc OK
```

Um processo faz referência a *y*, mas *x* não é lido por outro processo. O valor final de *x* pode ser 1 ou 2, e o valor final de *y* é sempre 1.

```
int x = 0; y = 0;  
co x = y + 1;  
// y = x + 1;  
oc NOK
```

Cada expressão possui uma referência crítica e cada processo atribui a uma variável lida pelo outro. Os valores de *x* e *y* podem ser 1 e 2, 2 e 1 ou 1 e 1 (inválido).

# Necessidade de sincronização

- Se expressão ou atribuição não satisfizer propriedade at-most-once, é necessário fazer com que ela execute atomicamente
- Genericamente, podemos precisar executar qualquer sequência de comandos como uma ação atômica
  - Precisamos de um mecanismo de sincronização para construir atomicidade grossa

# Condição de corrida e seção crítica

- Condição de corrida (*race condition*)
  - Situação onde diferentes processos acessam e manipulam o mesmo dado concorrentemente
    - Resultado da execução depende da **ordem em que os acessos acontecem**
    - Alguns ordenamentos geram resultados inconsistentes
  - Ocorre quando o resultado de uma computação varia de acordo com as velocidades (ou “*timing*”) dos processos → **imprevisível!!!**

| Operações    |              | Conflito | Motivo                                                                   |
|--------------|--------------|----------|--------------------------------------------------------------------------|
| <b>read</b>  | <b>read</b>  | não      | Operações de <b>read</b> independem da ordem em que são executadas       |
| <b>read</b>  | <b>write</b> | sim      | O valor do <b>read</b> depende se é feito antes ou depois de uma escrita |
| <b>write</b> | <b>write</b> | sim      | O valor final depende da ordem em que os <b>writes</b> são feitos        |

# Solução para problema seção crítica: exclusão mútua

- Garantir acesso exclusivo de um processo à seção crítica
  - Se um processo está executando instruções da seção crítica, outro processo é impedido de entrar até que o primeiro saia
    - É o que se denomina de exclusão mútua
  - Resulta em uma **serialização** no acesso
- Primitivas para exclusão mútua: **Enter\_EM** e **Exit\_EM**



# Primitiva de sincronização: await (Andrews)

- Notação:

`< await (B) S;>`

- Executa o comando S de forma indivisível (atômica) somente quando a expressão B (*boolean*) for verdadeira
- Condição para atrasar execução do processo especificada por condição B

- Propriedades:

- B garantidamente será true quando S começar a executar
- Nenhum estado interno em S é visível por outros processos

- Exemplo:

`< await (s > 0) s = s - 1; >`

- Espera até que s seja positivo, e então decrementa s
- Valor de s necessariamente será positivo antes de ser decrementado

# Primitiva de sincronização: await (Andrews)

- Forma genérica que expressa exclusão mútua e sinc. condicional
  - Ambos, simultaneamente ou independentemente
- Para expressar apenas:
  - Exclusão mútua: omite o *await* (B)
    - Operador de atomicidade é “<” e “>”

```
<x = x + 1; y = y + 1;>
```

- Estado intermediário onde x foi incrementado mas y não foi, por definição, não é visível por outros processos
- Sincronização condicional: omite o S;

```
<await (count > 0);>
```

- Atrasa execução do processo até que count > 0

Se a expressão B atender propriedade **at-most-once** é possível implementar **<await (B); >** simplesmente como **while (not B);**

# Exemplo: produtor/consumidor

- Situação comum em várias situações: buffer de comunicação, filas de impressão, etc...
- Princípio:
  - Produtor “produz” itens enquanto tiver capacidade de armazenamento.
    - Não pode exceder a sua capacidade
  - Consumidor “consome” itens enquanto houver disponibilidade
    - Não pode consumir o que não existe
- Variações:
  - n produtores, m consumidores, buffer unitário, buffer ilimitado, etc..
- Problema: **copiar todos os elementos** de um array, do produtor para o consumidor, utilizando um **buffer unitário** (int)
  - Buffer como variável compartilhada usada para comunicação entre P/C
  - Como sincronizar acesso ao buffer?

# Exemplo: produtor/consumidor

```
int buf, p = 0, c = 0;

process producer{
    int a[n];
    while (p < n) {
        <await (p == c);>
        buf = a[p];
        p = p + 1;
    }
}

process consumer{
    int b[n];
    while (c < n) {
        <await (p > c);>
        b[c] = buf;
        c = c + 1;
    }
}
```

→ p e c armazenam o número de itens produzidos e consumidos, respectivamente

→ while (not (p == c));

Já que B referencia uma variável alterada por outro processo (*at most once*)



*Spin lock ou busy waiting!!*



# Como implementar operações atômicas?

- Sem suporte de mecanismos de hardware, i.é., em software puro
  - Algoritmo de Dekker
  - Algoritmo de Peterson
  - Algoritmo de Lamport
- Com suporte de mecanismos de hardware
  - Baseado em habilitação/desabilitação de interrupções
  - Instruções atômicas (teste-and-set ou swap)

# Leituras adicionais

- Andrews, G. *“Foundations of Multithreaded, Parallel and Distributed Programming”*, Addison-Wesley, 2000.
  - Capítulo 2 (seções 2.1 a 2.5)

# Exercício

1. Considere os 3 comandos abaixo:

```
C1: x = x + y;  
C2: y = x - y;  
C3: x = x - y;
```

Assuma que x é inicialmente 2 e que y é inicialmente 5. Para cada um dos casos abaixo, quais são os valores finais possíveis para x e y? Explique.

- a) C<sub>1</sub>; C<sub>2</sub>; C<sub>3</sub>;
- b) co <C<sub>1</sub>> // <C<sub>2</sub>> // <C<sub>3</sub>> oc
- c) co <await (x > y) C<sub>1</sub>; C<sub>2</sub>> // <C<sub>3</sub>> oc